



Département de Mathématique et d'Informatique

Année universitaire 2026–2027

Gestion d'un système de réservation de salles

Modélisation UML et implémentation en C++

Réalisé par :

BOTON H. Désiré

Matricule : 11052STI23

13 novembre 2026

Table des matières

Introduction **3**

1 Analyse du besoin **4**
1.1 Acteurs . . . 4
1.2 Besoins fonctionnels . . . 4
1.3 Règles de gestion . . . 4

2 Identification des classes **5**

3 Modélisation UML **6**
3.1 Diagramme de cas d'utilisation . . . 6
3.2 Diagramme de classes . . . 7
3.3 Diagramme de séquence : réservation d'une salle . . . 7

4 Implémentation en C++ **9**
4.1 Choix de conception . . . 9
4.2 Code source . . . 9
4.2.1 Énumération `Categorie` et classe `Salle` . . . 9
4.2.2 Classe `Enseignant` . . . 10
4.2.3 Classe `Creneau` . . . 10
4.2.4 Classe `Reservation` et exception `ConflitReservation` . . . 11
4.2.5 Classe `GestionnaireReservations` . . . 12
4.2.6 Programme principal de démonstration . . . 14
4.3 Compilation et exécution . . . 15
4.4 Résultat attendu . . . 15
4.5 Tests des contraintes métier . . . 16

Conclusion **17**

A Sources PlantUML des diagrammes **18**
A.1 Cas d'utilisation . . . 18
A.2 Diagramme de classes . . . 18
A.3 Diagramme de séquence . . . 19

Table des figures

3.1 Diagramme de cas d'utilisation 6
3.2 Diagramme de classes UML 7
3.3 Diagramme de séquence : réservation d'une salle 8

Introduction

Un département universitaire dispose de plusieurs salles utilisées pour les cours, les travaux dirigés (TD) et les soutenances. Chaque salle est caractérisée par un numéro, une capacité et une catégorie. Les enseignants réservent ces salles pour des dates et des créneaux horaires précis.

La contrainte métier principale est la suivante : **une salle ne peut pas être réservée par deux enseignants pour le même créneau**.

Ce rapport présente la démarche de conception orientée objet : identification des classes, diagrammes UML (cas d'utilisation, classes, séquence), puis implémentation en C++ des fonctionnalités de vérification de disponibilité, d'enregistrement et d'annulation de réservations, avec rejet des réservations conflictuelles.

Chapitre 1

Analyse du besoin

1.1 Acteurs

— **Enseignant** : consulte les disponibilités, réserve une salle et annule ses propres réservations.
— **Administrateur** : gère le parc de salles (ajout, modification, suppression) et peut consulter toutes les réservations.

1.2 Besoins fonctionnels

1. Consulter la liste des salles et leurs caractéristiques.
2. Vérifier la disponibilité d'une salle pour une date et un créneau.
3. Enregistrer une réservation.
4. Annuler une réservation.
5. Consulter les réservations existantes.
6. Gérer les salles (administrateur).

1.3 Règles de gestion

R1 Une salle ne peut pas être réservée par deux enseignants pour deux créneaux qui se chevauchent le même jour.
R2 Un créneau est valide si l'heure de début est strictement inférieure à l'heure de fin.
R3 Le nombre de participants ne doit pas dépasser la capacité de la salle.
R4 Seul l'enseignant ayant effectué une réservation peut l'annuler.
R5 Une salle appartient à une seule catégorie : cours, travaux dirigés ou soutenance.

Chapitre 2

Identification des classes

L'analyse du texte (noms communs du domaine) conduit aux classes suivantes.

Classe	Attributs	Responsabilité
`Salle`	numéro, capacité, catégorie	Représenter une salle et ses caractéristiques.
`Categorie` (énumération)	COURS, TRAVAUX_DIRIGES, SOUTENANCE	Typer l'usage d'une salle.
`Enseignant`	identifiant, nom, prénom	Représenter l'utilisateur qui réserve.
`Creneau`	date, heure de début, heure de fin	Représenter une plage horaire et détecter les chevauchements.
`Reservation`	identifiant, nombre de participants, motif	Associer une salle, un enseignant et un créneau.
`GestionnaireReservations`	collections de salles, enseignants et réservations	Appliquer les règles métier : disponibilité, réservation, annulation.
`ConflitReservation` (exception)	message	Signaler une violation de la règle R1.

Table 2.1. Classes identifiées

Chapitre 3

Modélisation UML

3.1 Diagramme de cas d'utilisation

Le diagramme de la figure 3.1 présente les interactions entre les acteurs et le système.

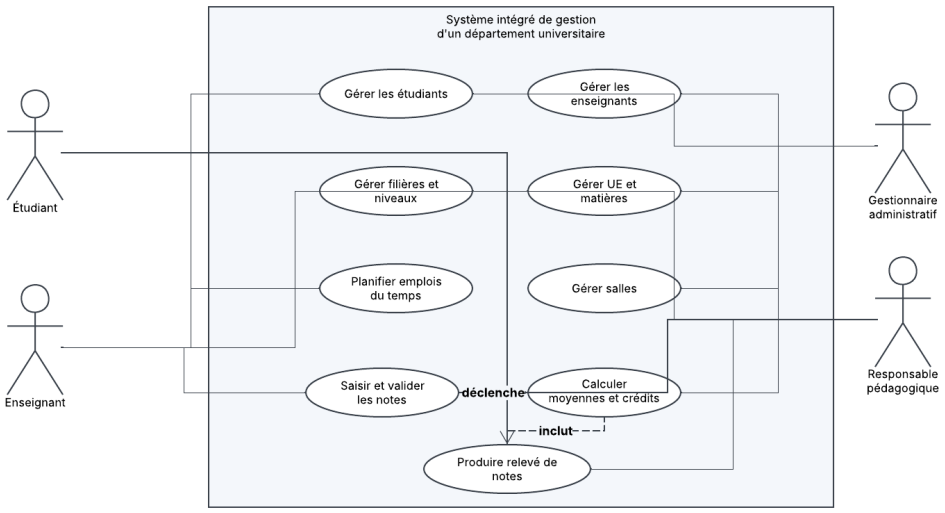


Figure 3.1. Diagramme de cas d'utilisation

Cas d'utilisation	Description
Consulter les salles	L'enseignant affiche les salles avec capacité et catégorie.
Vérifier la disponibilité	Le système indique si la salle est libre sur le créneau demandé.
Réserver une salle	Inclut « Vérifier la disponibilité » ; échoue en cas de conflit.
Annuler une réservation	Supprime une réservation appartenant à l'enseignant.
Gérer les salles	L'administrateur ajoute ou supprime des salles.

Table 3.1. Description des cas d'utilisation

3.2 Diagramme de classes

La figure 3.2 présente le diagramme de classes UML.

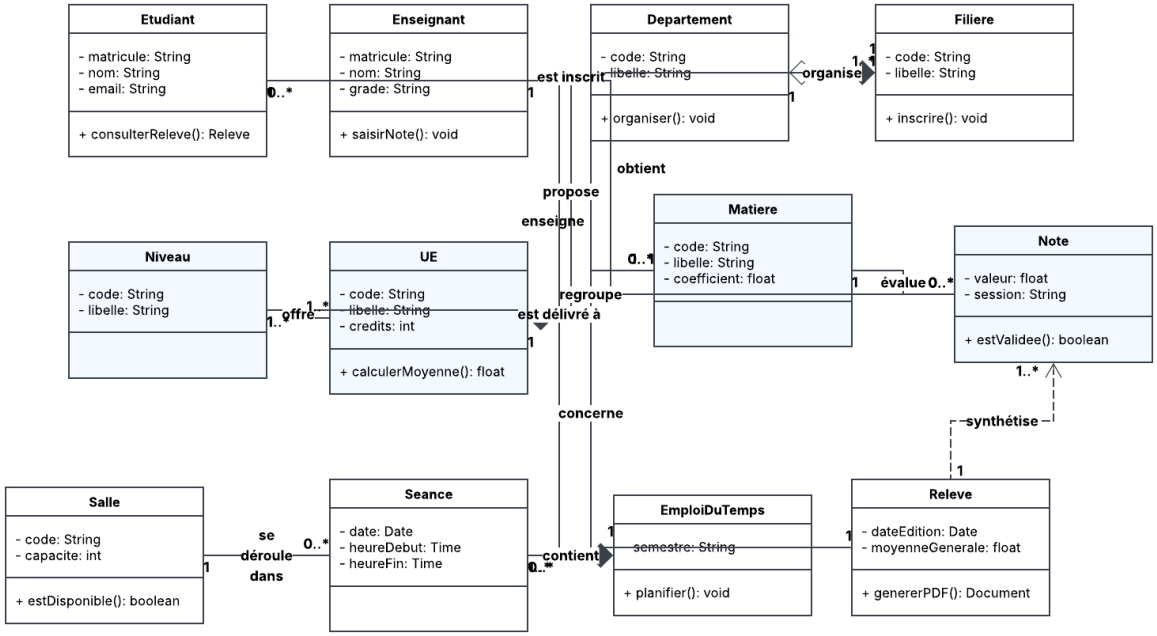


Figure 3.2. Diagramme de classes UML

Associations et multiplicités

- `Enseignant` **1** — **0..*** `Reservation` : un enseignant effectue zéro ou plusieurs réservations ; une réservation est effectuée par un seul enseignant.
- `Salle` **1** — **0..*** `Reservation` : une salle peut être réservée plusieurs fois (sur des créneaux différents) ; une réservation concerne une seule salle.
- `Reservation` **1** — **1** `Creneau` : composition ; le créneau n'existe que dans la réservation.
- `Salle` * — `Categorie` **1** : chaque salle a une unique catégorie.
- `GestionnaireReservations` **1** — **0..*** `Salle`, `Enseignant`, `Reservation` : le gestionnaire conserve les collections d'objets.

3.3 Diagramme de séquence : réservation d'une salle

La figure 3.3 illustre le scénario de réservation, avec le cas nominal et le cas de conflit.

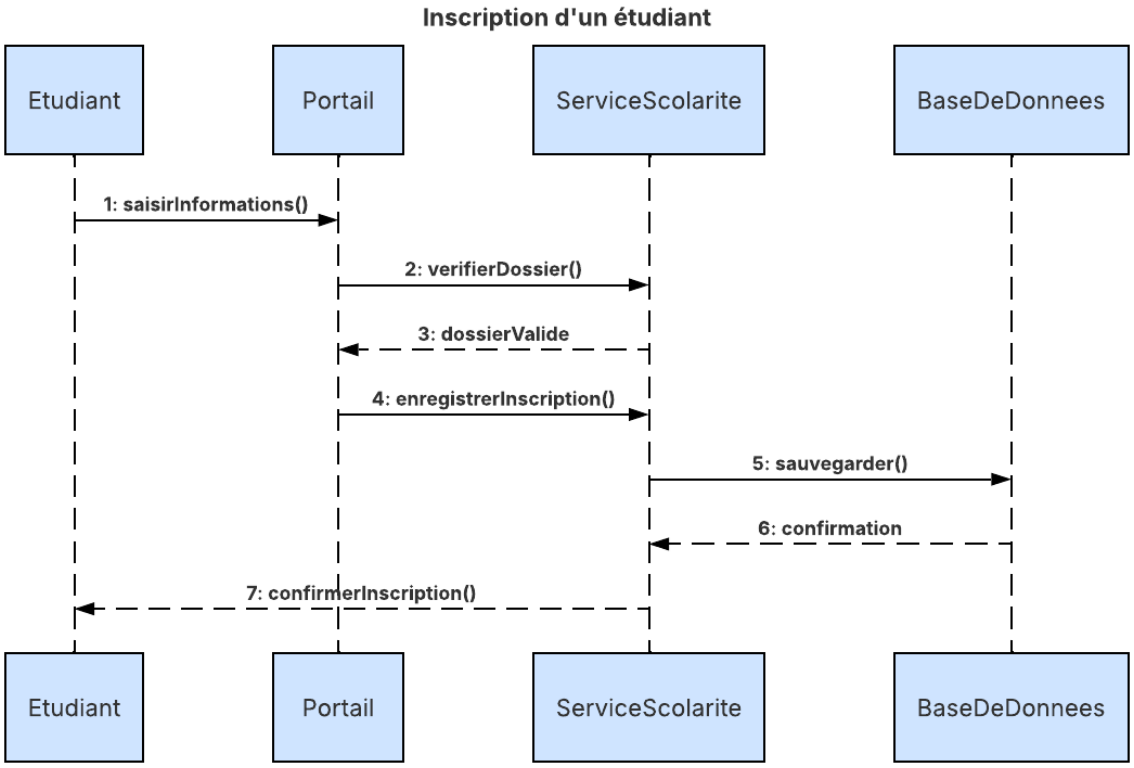


Figure 3.3. Diagramme de séquence : réservation d'une salle

Déroulement.

1. L'enseignant demande une réservation (salle, créneau, participants, motif).
2. Le gestionnaire recherche la salle et l'enseignant.
3. Il vérifie la capacité (R3).
4. Il parcourt les réservations de la salle et appelle `Creneau::chevauche()` (R1).
5. **Alternative** : si un chevauchement existe, une exception `ConflitReservation` est levée et rien n'est enregistré ; sinon, la réservation est créée, ajoutée à la collection et son identifiant est retourné.

Chapitre 4

Implémentation en C++

4.1 Choix de conception

— **Encapsulation** : tous les attributs sont privés, accessibles par des accesseurs ; les invariants (créneau valide, capacité positive) sont vérifiés dans les constructeurs.
— **Collections** : `std::vector` de `std::shared_ptr` pour les salles, enseignants et réservations.
— **Contraintes métier** : centralisées dans `GestionnaireReservations` et signalées par des exceptions.
— **Détection de chevauchement** : deux créneaux $[d_1, f_1[$ et $[d_2, f_2[$ du même jour se chevauchent si et seulement si $d_1 < f_2$ et $d_2 < f_1$. Ainsi, deux créneaux consécutifs (10h–12h et 12h–14h) ne sont pas en conflit.

4.2 Code source

4.2.1 Énumération `Categorie` et classe `Salle`

```cpp
#include <algorithm>
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>

enum class Categorie { COURS, TRAVAUX_DIRIGES, SOUTENANCE };

inline std::string categorieVersTexte(Categorie c) {
    switch (c) {
        case Categorie::COURS:           return "Cours";
        case Categorie::TRAVAUX_DIRIGES: return "Travaux diriges";
        case Categorie::SOUTENANCE:      return "Soutenance";
    }
    return "Inconnue";
}

class Salle {
private:
    std::string numero;
    int capacite;
```

```
    Categorie categorie;

public:
    Salle(const std::string& numero, int capacite, Categorie categorie)
        : numero(numero), capacite(capacite), categorie(categorie) {
        if (numero.empty())
            throw std::invalid_argument("Numero de salle vide");
        if (capacite <= 0)
            throw std::invalid_argument("Capacite invalide");
    }

    const std::string& getNumero() const { return numero; }
    int getCapacite() const { return capacite; }
    Categorie getCategorie() const { return categorie; }

    void afficher() const {
        std::cout << "Salle " << numero << " | capacite : " << capacite
                  << " | categorie : " << categorieVersTexte(categorie)
                     << "\n";
    }
};
```

4.2.2 Classe Enseignant

```
class Enseignant {
private:
    int id;
    std::string nom;
    std::string prenom;

public:
    Enseignant(int id, const std::string& nom, const std::string& prenom
        )
        : id(id), nom(nom), prenom(prenom) {}

    int getId() const { return id; }
    const std::string& getNom() const { return nom; }
    const std::string& getPrenom() const { return prenom; }
    std::string nomComplet() const { return prenom + " " + nom; }
};
```

4.2.3 Classe Creneau

```
class Creneau {
private:
    std::string date;    // format AAAA-MM-JJ
    int heureDebut;      // 0..24
    int heureFin;        // 0..24
```

```
6
7  public:
8      Creneau(const std::string& date, int heureDebut, int heureFin)
9          : date(date), heureDebut(heureDebut), heureFin(heureFin) {
10         if (heureDebut < 0 || heureFin > 24 || heureDebut >= heureFin)
11             throw std::invalid_argument("Creneau invalide");
12     }
13
14     const std::string& getDate() const { return date; }
15     int getHeureDebut() const { return heureDebut; }
16     int getHeureFin() const { return heureFin; }
17
18     // Vrai si les deux creneaux ont lieu le meme jour et se superposent
19     bool chevauche(const Creneau& autre) const {
20         return date == autre.date &&
21                heureDebut < autre.heureFin &&
22                autre.heureDebut < heureFin;
23     }
24
25     std::string texte() const {
26         return date + " " + std::to_string(heureDebut) + "h-" +
27                std::to_string(heureFin) + "h";
28     }
29 };
```

4.2.4 Classe `Reservation` et exception `ConflitReservation`

```
1  class ConflitReservation : public std::runtime_error {
2  public:
3      explicit ConflitReservation(const std::string& msg)
4          : std::runtime_error(msg) {}
5  };
6
7  class Reservation {
8  private:
9      int id;
10     std::shared_ptr<Salle> salle;
11     std::shared_ptr<Enseignant> enseignant;
12     Creneau creneau;
13     int nbParticipants;
14     std::string motif;
15
16 public:
17     Reservation(int id, std::shared_ptr<Salle> salle,
18                 std::shared_ptr<Enseignant> enseignant,
19                 const Creneau& creneau, int nbParticipants,
20                 const std::string& motif)
21         : id(id), salle(salle), enseignant(enseignant), creneau(creneau)
              ,
```

```
22          nbParticipants(nbParticipants), motif(motif) {}
23
24      int getId() const { return id; }
25      const Salle& getSalle() const { return *salle; }
26      const Enseignant& getEnseignant() const { return *enseignant; }
27      const Creneau& getCreneau() const { return creneau; }
28      int getNbParticipants() const { return nbParticipants; }
29      const std::string& getMotif() const { return motif; }
30
31      void afficher() const {
32          std::cout << "Reservation #" << id << " | salle "
33                    << salle->getNumero() << " | "
34                    << enseignant->nomComplet() << " | "
35                    << creneau.texte() << " | " << motif << "\n";
36      }
37  };
```

4.2.5 Classe GestionnaireReservations

```
1   class GestionnaireReservations {
2   private:
3       std::vector<std::shared_ptr<Salle>> salles;
4       std::vector<std::shared_ptr<Enseignant>> enseignants;
5       std::vector<Reservation> reservations;
6       int prochainId = 1;
7
8       std::shared_ptr<Salle> trouverSalle(const std::string& numero) const
            {
9           for (const auto& s : salles)
10              if (s->getNumero() == numero) return s;
11          throw std::invalid_argument("Salle introuvable : " + numero);
12      }
13
14      std::shared_ptr<Enseignant> trouverEnseignant(int id) const {
15          for (const auto& e : enseignants)
16              if (e->getId() == id) return e;
17          throw std::invalid_argument("Enseignant introuvable");
18      }
19
20  public:
21      void ajouterSalle(const Salle& s) {
22          for (const auto& existante : salles)
23              if (existante->getNumero() == s.getNumero())
24                  throw std::invalid_argument("Numero de salle deja
                        utilise");
25          salles.push_back(std::make_shared<Salle>(s));
26      }
27
28      void ajouterEnseignant(const Enseignant& e) {
```

```
29          enseignants.push_back(std::make_shared<Enseignant>(e));
30      }
31
32      // Verifie la disponibilite d'une salle pour un creneau
33      bool estDisponible(const std::string& numeroSalle,
34                         const Creneau& creneau) const {
35          trouverSalle(numeroSalle); // leve une exception si inexistante
36          for (const auto& r : reservations)
37              if (r.getSalle().getNumero() == numeroSalle &&
38                  r.getCreneau().chevauche(creneau))
39                  return false;
40          return true;
41      }
42
43      // Enregistre une reservation ; retourne son identifiant
44      int reserver(const std::string& numeroSalle, int idEnseignant,
45                   const Creneau& creneau, int nbParticipants,
46                   const std::string& motif) {
47          auto salle = trouverSalle(numeroSalle);
48          auto enseignant = trouverEnseignant(idEnseignant);
49
50          if (nbParticipants <= 0 || nbParticipants > salle->getCapacite()
                )
51              throw std::invalid_argument(
52                  "Nombre de participants superieur a la capacite");
53
54          if (!estDisponible(numeroSalle, creneau))
55              throw ConflitReservation("La salle " + numeroSalle +
56                  " est deja reservee sur le creneau " + creneau.texte());
57
58          reservations.emplace_back(prochainId, salle, enseignant,
59                                    creneau, nbParticipants, motif);
60          return prochainId++;
61      }
62
63      // Annule une reservation (seul son auteur peut l'annuler)
64      void annuler(int idReservation, int idEnseignant) {
65          auto it = std::find_if(reservations.begin(), reservations.end(),
66              [&](const Reservation& r) { return r.getId() ==
                    idReservation; });
67
68          if (it == reservations.end())
69              throw std::invalid_argument("Reservation introuvable");
70          if (it->getEnseignant().getId() != idEnseignant)
71              throw std::invalid_argument(
72                  "Seul l'auteur peut annuler la reservation");
73
74          reservations.erase(it);
75      }
76
77      void afficherSalles() const {
```

```
78          for (const auto& s : salles) s->afficher();
79      }
80
81      void afficherReservations() const {
82          if (reservations.empty()) std::cout << "Aucune reservation.\n";
83          for (const auto& r : reservations) r.afficher();
84      }
85  };
```

4.2.6 Programme principal de démonstration

```
1   int main() {
2       GestionnaireReservations g;
3
4       g.ajouterSalle(Salle("A101", 60, Categorie::COURS));
5       g.ajouterSalle(Salle("B202", 30, Categorie::TRAVAUX_DIRIGES));
6       g.ajouterSalle(Salle("S01", 20, Categorie::SOUTENANCE));
7
8       g.ajouterEnseignant(Enseignant(1, "Martin", "Alice"));
9       g.ajouterEnseignant(Enseignant(2, "Durand", "Paul"));
10
11      std::cout << "--- Salles ---\n";
12      g.afficherSalles();
13
14      try {
15          // 1. Reservation valide
16          int r1 = g.reserver("A101", 1, Creneau("2026-11-20", 8, 10),
17                              45, "Cours de POO");
18          std::cout << "\nReservation " << r1 << " enregistree.\n";
19
20          // 2. Conflit : meme salle, creneau qui chevauche, autre
                enseignant
21          g.reserver("A101", 2, Creneau("2026-11-20", 9, 11),
22                     40, "Cours d'algorithmique");
23      } catch (const ConflitReservation& e) {
24          std::cout << "[CONFLIT] " << e.what() << "\n";
25      }
26
27      try {
28          // 3. Creneau consecutif : pas de conflit
29          int r2 = g.reserver("A101", 2, Creneau("2026-11-20", 10, 12),
30                              40, "Cours d'algorithmique");
31          std::cout << "Reservation " << r2 << " enregistree.\n";
32
33          // 4. Capacite depassee
34          g.reserver("S01", 1, Creneau("2026-11-21", 14, 16),
35                     50, "Soutenance");
36      } catch (const std::exception& e) {
37          std::cout << "[ERREUR] " << e.what() << "\n";
```

```
38      }
39
40      std::cout << "\n--- Reservations ---\n";
41      g.afficherReservations();
42
43      // 5. Annulation puis nouvelle verification de disponibilite
44      g.annuler(1, 1);
45      std::cout << "\nReservation 1 annulee.\n";
46      std::cout << "A101 libre le 2026-11-20 de 8h a 10h ? "
47                << (g.estDisponible("A101", Creneau("2026-11-20", 8, 10))
48                    ? "oui" : "non") << "\n";
49      return 0;
50  }
```

4.3 Compilation et exécution

```
g++ -std=c++17 -Wall -Wextra -o reservation main.cpp
./reservation
```

4.4 Résultat attendu

```
--- Salles ---
Salle A101 | capacite : 60 | categorie : Cours
Salle B202 | capacite : 30 | categorie : Travaux diriges
Salle S01 | capacite : 20 | categorie : Soutenance

Reservation 1 enregistree.
[CONFLIT] La salle A101 est deja reservee sur le creneau 2026-11-20 9h
    -11h
Reservation 2 enregistree.
[ERREUR] Nombre de participants superieur a la capacite

--- Reservations ---
Reservation #1 | salle A101 | Alice Martin | 2026-11-20 8h-10h | Cours
    de POO
Reservation #2 | salle A101 | Paul Durand | 2026-11-20 10h-12h | Cours d
    'algorithmique

Reservation 1 annulee.
A101 libre le 2026-11-20 de 8h a 10h ? oui
```

4.5 Tests des contraintes métier

Scénario	Règle	Résultat
Même salle, créneaux qui se chevauchent (8h–10h et 9h–11h)	R1	Exception `ConflitReservation`
Même salle, créneaux consécutifs (8h–10h et 10h–12h)	R1	Réservation acceptée
Même salle, jours différents	R1	Réservation acceptée
50 participants dans une salle de 20 places	R3	`invalid_argument`
Annulation par un autre enseignant	R4	`invalid_argument`
Début $\geq$ fin	R2	`invalid_argument`

Table 4.1. Jeux de tests

Conclusion

Ce travail a permis de modéliser et d'implémenter un système de réservation de salles. L'identification des classes et leurs associations (avec multiplicités) a été formalisée par trois diagrammes UML, puis traduite en C++ en respectant l'encapsulation et en utilisant des collections d'objets.

La contrainte principale, l'interdiction des réservations conflictuelles, est garantie par la méthode `Creneau::chevauche()` et par le contrôle effectué dans `GestionnaireReservations::reserve`

Parmi les évolutions possibles : persistance des données (fichier ou base de données), gestion de réservations récurrentes, interface graphique, et utilisation d'un type `std::chrono` pour les dates.

Annexe A

Sources PlantUML des diagrammes

Ces sources permettent de générer les images à insérer dans le rapport (site https://www.plantuml.com/plantuml ou extension PlantUML ou https://lucid.app/(Outile en ligne)). Enregistrer les images dans le dossier `images/` sous les noms `cas_utilisation.png`, `diagramme_classes.png` et `diagramme_sequence.png`.

A.1 Cas d'utilisation

```
@startuml
left to right direction
actor Enseignant
actor Administrateur
rectangle "Systeme de reservation" {
  usecase "Consulter les salles" as UC1
  usecase "Verifier la disponibilite" as UC2
  usecase "Reserver une salle" as UC3
  usecase "Annuler une reservation" as UC4
  usecase "Consulter les reservations" as UC5
  usecase "Gerer les salles" as UC6
}
Enseignant --> UC1
Enseignant --> UC2
Enseignant --> UC3
Enseignant --> UC4
Enseignant --> UC5
Administrateur --> UC5
Administrateur --> UC6
UC3 .> UC2 : <<include>>
@enduml
```

A.2 Diagramme de classes

```
@startuml
enum Categorie {
  COURS
  TRAVAUX_DIRIGES
  SOUTENANCE
}
```

```
class Salle {
  -numero : string
  -capacite : int
  -categorie : Categorie
  +getNumero() : string
  +getCapacite() : int
  +getCategorie() : Categorie
}
class Enseignant {
  -id : int
  -nom : string
  -prenom : string
  +getId() : int
  +nomComplet() : string
}
class Creneau {
  -date : string
  -heureDebut : int
  -heureFin : int
  +chevauche(autre : Creneau) : bool
}
class Reservation {
  -id : int
  -nbParticipants : int
  -motif : string
}
class GestionnaireReservations {
  +estDisponible(numero, creneau) : bool
  +reserver(numero, idEns, creneau, nb, motif) : int
  +annuler(idReservation, idEns) : void
}
class ConflitReservation
Salle "*" --> "1" Categorie
Enseignant "1" -- "0..*" Reservation : effectue
Salle "1" -- "0..*" Reservation : concerne
Reservation "1" *-- "1" Creneau
GestionnaireReservations "1" o-- "0..*" Salle
GestionnaireReservations "1" o-- "0..*" Enseignant
GestionnaireReservations "1" o-- "0..*" Reservation
GestionnaireReservations ..> ConflitReservation : <<throws>>
@enduml
```

A.3 Diagramme de séquence

```
@startuml
actor Enseignant
participant ":GestionnaireReservations" as G
participant ":Salle" as S
participant ":Reservation" as R
```

```
participant ":Creneau" as C

Enseignant -> G : reserver(numero, idEns, creneau, nb, motif)
activate G
G -> G : trouverSalle(numero)
G -> S : getCapacite()
S --> G : capacite
G -> G : verifier capacite
G -> G : estDisponible(numero, creneau)
loop pour chaque reservation de la salle
  G -> C : chevauche(autre)
  C --> G : vrai / faux
end
alt conflit detecte
  G --> Enseignant : ConflitReservation
else salle disponible
  G -> R ** : create(id, salle, enseignant, creneau, ...)
  G -> G : ajouter a la collection
  G --> Enseignant : idReservation
end
deactivate G
@enduml
```